

Control de un Drone Tello mediante ROS y Python

Mateo Eduardo Bermeo Pesántez
Facultad de Ingeniería - Telecomunicaciones
Universidad de Cuenca
Cuenca, Ecuador
mateo.bermeop@ucuenca.edu.ec

Santiago Andrés Guillén Malla
Facultad de Ingeniería - Telecomunicaciones
Universidad de Cuenca
Cuenca, Ecuador
santiago.guillenm@ucuenca.edu.ec

Vicente Paúl Jiménez Ávila
Facultad de Ingeniería - Telecomunicaciones
Universidad de Cuenca
Cuenca, Ecuador
paul.jimenez@ucuenca.edu.ec

Resumen—Este proyecto presenta la implementación de un sistema de control, monitoreo y procesamiento de video para el dron DJI Tello utilizando el marco robótico ROS 2 Jazzy, ejecutado dentro de un entorno aislado mediante Docker. Se desarrollaron seis nodos ROS2 en Python para gestionar la conexión con el dron, la ejecución de misiones autónomas, la recepción de telemetría, la supervisión del nivel de batería, la visualización del video transmitido y la detección de objetos por colores. Para garantizar reproducibilidad, se construyó una imagen Docker personalizada con ROS2, OpenCV, cv_bridge y la librería djitellopy, permitiendo la interacción directa con el dron a través de su red Wi-Fi. El sistema integra tópicos, publicadores y suscriptores para coordinar las funciones del dron en tiempo real, y se evaluó mediante pruebas prácticas de vuelo, detección visual y monitoreo de recursos del contenedor. Los resultados demuestran la efectividad del enfoque modular con ROS2 para el desarrollo de aplicaciones aéreas distribuidas basadas en robots móviles.

Index Terms—ROS2, Drone Tello, Python, Comunicación inalámbrica, Nodos ROS, Video streaming.

I. INTRODUCCIÓN

El uso de robots aéreos de bajo costo, como el **DJI Tello**, se ha vuelto común en entornos académicos debido a su facilidad de programación y su compatibilidad con bibliotecas modernas de visión computacional y control. En paralelo, **ROS 2** se ha consolidado como el estándar para el desarrollo de sistemas robóticos distribuidos, gracias a su arquitectura basada en nodos, tópicos y mecanismos de comunicación de baja latencia.

En este proyecto se integró el dron Tello dentro de un ecosistema completo de ROS 2, ejecutado en un contenedor Docker especialmente configurado para incluir ROS2 Jazzy, OpenCV, cv_bridge y la librería djitellopy. El uso de Docker permitió asegurar un entorno controlado, portable y libre de conflictos con el sistema host, mientras que ROS 2 proporcionó la infraestructura de comunicación necesaria para

distribuir las tareas entre varios nodos especializados.

Se implementaron seis nodos ROS2 en Python, cada uno con una responsabilidad clara: conexión y control del dron, planificación de misiones automáticas, monitoreo de telemetría, supervisión de batería, visualización del flujo de video y detección de objetos de color rojo y negro mediante procesamiento de imágenes. La arquitectura final se validó ejecutando pruebas reales de vuelo, análisis visual y registro de datos operativos, demostrando la robustez del enfoque modular y la utilidad de ROS2 para aplicaciones de robótica aérea.

II. MARCO TEÓRICO

II-A. Robot Operating System 2 (ROS2)

ROS 2 es un framework para el desarrollo de software robótico distribuido, diseñado sobre un modelo de comunicación basado en *publishers*, *subscribers* y *services*. A diferencia de ROS 1, ROS 2 utiliza el protocolo DDS (Data Distribution Service), lo que permite comunicación en tiempo real, soporte nativo para sistemas distribuidos, y mayor confiabilidad en redes inalámbricas [1], [2]. En este proyecto, ROS 2 permitió ejecutar seis nodos independientes encargados de diferentes procesos del dron Tello, garantizando modularidad, paralelismo y facilidad de depuración.

II-B. Dron DJI Tello y protocolo de comunicación

El **DJI Tello** es un micro-UAV diseñado para aplicaciones educativas y de experimentación. Su comunicación se realiza por **UDP** sobre Wi-Fi:

- Puerto 8889: comandos de control
- Puerto 8890: telemetría
- Puerto 11111: transmisión de video H.264

El SDK del Tello define un conjunto de comandos simples en texto plano (*takeoff*, *land*, *forward 50*, etc.), los cuales pueden ejecutarse mediante sockets UDP o mediante librerías como *djitellopy*, usada en este proyecto [3]. Este protocolo posibilita una comunicación de baja latencia, adecuada para control reactivo y transmisión continua de datos.

II-C. Uso de Docker en entornos robóticos

Docker permite encapsular aplicaciones en contenedores aislados que incluyen todas sus dependencias. En robótica, su uso se ha extendido para evitar problemas comunes de incompatibilidad entre versiones de ROS, librerías de visión o compiladores [4].

En este trabajo, se construyó una imagen Docker personalizada que integra:

- ROS 2 Jazzy
- OpenCV y *cv_bridge*
- *djitellopy*
- utilidades X11 para visualización

Este enfoque asegura que el sistema funcione igual en cualquier computadora, lo que es crucial para experimentación con hardware real.

II-D. Procesamiento de video en tiempo real

El dron Tello transmite video en formato **H.264**, que puede ser decodificado mediante **OpenCV**. En ROS2, las imágenes se publican en mensajes del tipo `sensor_msgs/Image`, permitiendo que diferentes nodos procesen el video de manera simultánea. La detección de objetos en este proyecto se realizó mediante segmentación por color en espacio HSV, extracción de contornos y filtrado por área mínima, una técnica clásica en visión computacional [5].

Este tipo de procesamiento permite desarrollar aplicaciones como conteo de objetos, navegación reactiva o visión basada en aprendizaje profundo.

II-E. Arquitectura distribuida basada en nodos

ROS 2 estructura el software como una red de nodos que se comunican mediante tópicos asíncronos. Esto facilita dividir un sistema complejo en componentes pequeños y manejables. Los seis nodos implementados son un ejemplo de arquitectura distribuida, en la cual tareas como control, video, telemetría y seguridad operan de manera paralela. Esto mejora la tolerancia a fallos y hace posible agregar nuevos módulos sin modificar el sistema central [6].

III. DESARROLLO

El desarrollo del proyecto se estructuró en cinco fases principales: (1) configuración del entorno de trabajo con Docker, (2) construcción de la imagen base con ROS 2 y librerías necesarias, (3) creación de los seis nodos ROS2,

(4) compilación e integración del workspace, y (5) ejecución y pruebas con el dron Tello. Cada fase se documenta a continuación en orden cronológico, siguiendo la secuencia mostrada en el documento original.

III-A. Fase 1: Configuración del entorno en Docker

La primera etapa consistió en preparar el sistema operativo Ubuntu para soportar Docker y Docker Compose. Se agregaron los permisos de administrador al usuario y se instaló `docker-compose-plugin`. Las Figuras 1 y 2 corresponden a las capturas donde se ejecutan los comandos iniciales:

```

paul@paul-VirtualBox:~$ sudo apt install ca-certificates curl gnupg -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
ca-certificates ya está en su versión más reciente (20240203-22.04.1).
gnupg ya está en su versión más reciente (2.2.27-ubuntu24.4).
El paquete curl no está instalado.
Se instalarán los siguientes paquetes NUEVOS:
  curl
0 actualizados, 1 nuevos se instalarán, 0 para eliminar y 93 no actualizados.
Se necesita descargar 194 kB de archivos.
Se utilizarán 455 kB de espacio de disco adicional después de esta operación.
Des11 http://ec.archive.ubuntu.com/ubuntu jammy-updates/main amd64 curl amd64 7.81.0-1ubuntu2.21 [194 kB]
Descargados 194 kB en 1s (164 kB/s)
Seleccionando el paquete curl previamente no seleccionado.
Leyendo la base de datos ... 249624 ficheros o directorios instalados actualmente.
Preparando para desempaquetar .../curl.7.81.0-1ubuntu2.21.amd64.deb ...
Desempaquetando curl (7.81.0-1ubuntu2.21) ...
Configurando curl (7.81.0-1ubuntu2.21) ...
Procesando disparadores para man-db (2.10.2-1) ...

```

Figura 1: Instalación de certificados, curl y claves GPG para Docker.

```

paul@paul-VirtualBox:~$ sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
paul@paul-VirtualBox:~$
paul@paul-VirtualBox:~$ echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
sudo tee /etc/apt/sources.list.d/docker.list && /dev/null
paul@paul-VirtualBox:~$
paul@paul-VirtualBox:~$ sudo apt update
Obj1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Obj2 http://ec.archive.ubuntu.com/ubuntu jammy InRelease
Obj3 https://download.docker.com/linux/ubuntu jammy InRelease [40.5 kB]
Obj4 http://ec.archive.ubuntu.com/ubuntu jammy-updates InRelease
Obj5 http://ec.archive.ubuntu.com/ubuntu jammy-backports InRelease
Obj6 https://download.docker.com/linux/ubuntu jammy/stable amd64 Packages [58.1 kB]
Descargados 187 kB en 3s (35.0 kB/s)
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 93 paquetes. Ejecute «apt list --upgradable» para verlos.
paul@paul-VirtualBox:~$ sudo apt install docker-compose-plugin -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  docker-buildx-plugin
Se instalarán los siguientes paquetes NUEVOS:
  docker-buildx-plugin docker-compose-plugin
0 actualizados, 2 nuevos se instalarán, 0 para eliminar y 93 no actualizados.
Se necesita descargar 30.2 MB de archivos.
Se utilizarán 155 MB de espacio de disco adicional después de esta operación.
Des11 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-buildx-plugin amd64 0.29.1-1-ubuntu.22.04-jammy [15.9 kB]
Des12 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-compose-plugin amd64 2.40.2-1-ubuntu.22.04-jammy [14.3 MB]
Descargados 30.2 MB en 8s (3.835 kB/s)
Seleccionando el paquete docker-buildx-plugin previamente no seleccionado.
(Leyendo la base de datos ... 249624 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../docker-buildx-plugin_0.29.1-1-ubuntu.22.04-jammy_amd64.deb ...
Desempaquetando docker-buildx-plugin (0.29.1-1-ubuntu.22.04-jammy) ...
Seleccionando el paquete docker-compose-plugin previamente no seleccionado.
Preparando para desempaquetar .../docker-compose-plugin_2.40.2-1-ubuntu.22.04-jammy_amd64.deb ...

```

Figura 2: Habilitación del repositorio oficial de Docker e instalación del plugin Compose.

Con Docker instalado, se verificó su funcionamiento mediante:

```
docker compose version
```

III-B. Fase 2: Creación del workspace y construcción de la imagen Docker

Se creó el workspace principal en el directorio del proyecto:

```
mkdir -p ./ros2_tello/ros2_ws/src
```

La Figura 3 muestra la estructura inicial del proyecto.

```

paul@paul-VirtualBox:~/Documents/Redes_Sensores/Proyecto$ tree ros2_tello/
ros2_tello/
├── ros2_ws
└── src

2 directories, 0 files
paul@paul-VirtualBox:~/Documents/Redes_Sensores/Proyecto$

```

Figura 3: Estructura del workspace `ros2_tello`.

Luego, se creó un archivo Dockerfile diseñado para incluir:

- ROS 2 Jazzy
- OpenCV y cv_bridge
- djitellopy
- entorno virtual Python
- soporte X11 para visualización

El Pseudocódigo 1 resume la lógica del Dockerfile.

Algorithm 1 Lógica general del Dockerfile

- 1: Instalar ROS 2 y dependencias del sistema
- 2: Crear entorno virtual Python
- 3: Instalar djitellopy, OpenCV y numpy
- 4: Crear workspace ROS2 vacío
- 5: Configurar variables de entorno y entrypoint

Una vez definido el archivo, la imagen se construyó con:

```
sudo docker build -t ros2_tello_image .
```

La Figura 4 muestra la salida durante el proceso de compilación.

```

paul@paul-VirtualBox: ~/Documents/rodes_sensores/Proyectos/ros2_tello$ docker build -t ros2_tello_image .
[+] Building 342.2s (40/46) FINISHED
=> [internal] load build definition from Dockerfile
=> [internal] load metadata for docker.io/ros/jazzy-desktop
=> [internal] load dockerignore
=> [internal] load context: .
=> [1/6] FROM docker.io/ros/jazzy-desktop
=> [2/6] WORKDIR /root
=> [3/6] RUN apt update && apt install -y python3-pip python3-colcon-common-extensions
=> [4/6] RUN mkdir -p /root/ros2_ws/src
=> [5/6] WORKDIR /root/ros2_ws
=> [6/6] RUN echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc && echo "source ~/ros2_ws/lo
=> exporting to image
=> exporting layers
=> writing image sha256:51a92c77b955aabb5d3504939089cb7b6a19ed401c0cb435e14de402676
=> naming to docker.io/library/ros2_tello_image

```

Figura 4: Construcción de la imagen Docker ros2_tello_image.

Posteriormente, se creó el archivo docker-compose.yml que define el contenedor con acceso a red en modo host, así como el montaje del workspace desde el host.

El contenedor se levantó con:

```
sudo docker compose up -d
```

La Figura 5 muestra la verificación con docker ps.

```

paul@paul-VirtualBox: ~/Documents/rodes_sensores/Proyectos/ros2_tello$ docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED        STATUS        PORTS        NAMES
7c08f9eb40     ros2_tello_image:latest   /ros_entrypoint.sh -   About a minute ago   Up about a minute   8889/tcp, 8890/tcp, 11111/tcp   ros2_tello

```

Figura 5: Contenedor ROS2 Tello ejecutándose en segundo plano.

III-C. Fase 3: Creación de los seis nodos ROS2

Dentro del contenedor, se creó el paquete principal:

```
ros2 pkg create --build-type ament_python
tello_control
```

La Figura 6 muestra la estructura generada del paquete.

```

root@ros2-tello:~/src# tree
.
├── tello_control
│   ├── LICENSE
│   ├── package.xml
│   ├── tello_control
│   ├── setup.cfg
│   ├── setup.py
│   ├── test.py
│   ├── test_copyright.py
│   ├── test_flake8.py
│   └── test_pep257.py
└── .gitignore

```

Figura 6: Estructura del paquete tello_control.

III-D. Esquema general de la arquitectura de conexión del sistema

La Figura 7 muestra el esquema completo de comunicación entre los componentes del sistema. El flujo incluye tres niveles:

- **Nivel 1 – Contenedor Docker:** Ejecución del entorno ROS2, nodos Python, librerías de visión y drivers de comunicación.
- **Nivel 2 – Nodos ROS2:** Cada nodo publica o se suscribe a tópicos específicos para control, telemetría, video y detección.
- **Nivel 3 – Dron DJI Tello:** Comunicación mediante WiFi (UDP) en puertos 8889, 8890 y 11111.

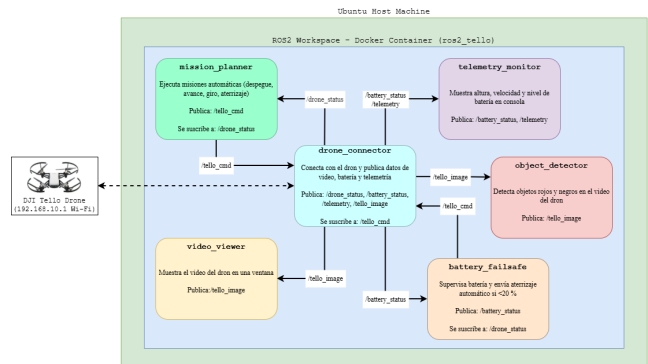


Figura 7: Esquema general de la arquitectura de conexión entre Docker, ROS2 y el dron Tello.

Este diagrama resume cómo el sistema opera de forma distribuida dentro del contenedor, mientras que la comunicación con el dron se mantiene directa por red en modo host.

A continuación, se desarrollaron los seis nodos principales del sistema, cada uno implementado en Python dentro del workspace de ROS2. Su función se resume a continuación:

- **Nodo 1: drone_connector.py** — conexión, telemetría, batería y video.
- **Nodo 2: mission_planner.py** — secuencia automática de despegue y movimientos.
- **Nodo 3: telemetry_monitor.py** — monitoreo continuo de batería, velocidad y altura.
- **Nodo 4: battery_failsafe.py** — aterrizaje automático por batería baja.
- **Nodo 5: video_viewer.py** — visualización del video recibido desde el dron.

- Nodo 6: **object_detector.py** — detección y conteo de objetos rojo/negro mediante procesamiento de imagen.

Cada uno de estos nodos se describe mediante un pseudocódigo representativo, donde se detalla su funcionamiento lógico antes de su implementación final en Python. Sus respectivos códigos completos se incluyen en los Anexos como los Listados 1–6.

Nodo 1: drone_connector.py: El pseudocódigo del Nodo 1 se presenta en el Algoritmo 2. Este nodo es responsable de establecer la conexión con el dron, publicar telemetría, procesar imágenes y reenviar comandos recibidos desde ROS2.

Algorithm 2 Flujo del Nodo 1: drone_connector.py

```

1: Conectar con el dron mediante djiellopy
2: Inicializar publicadores de batería, altura, telemetría y video
3: Suscribirse al tópico /tello/cmd
4: while ROS2 activo do
5:   Leer valores de batería, altura y frame de video
6:   Publicar telemetría en sus respectivos tópicos
7:   Recibir comandos enviados desde ROS2 y transmitirlos al dron
8: end while

```

Nodo 2: mission_planner.py: El Nodo 2 ejecuta una misión programada automáticamente. Su funcionamiento se resume en el Algoritmo 3.

Algorithm 3 Flujo del Nodo 2: mission_planner.py

```

1: Publicar comando takeoff
2: Esperar intervalo de seguridad
3: Enviar secuencia de movimientos (avanzar, girar, avanzar)
4: Publicar comando land

```

Nodo 3: telemetry_monitor.py: Este nodo imprime en consola la telemetría recibida desde el Nodo 1. Su lógica se detalla en el Algoritmo 4.

Algorithm 4 Flujo del Nodo 3: telemetry_monitor.py

```

1: while ROS2 activo do
2:   Leer datos de batería, velocidad y altura desde los tópicos
3:   Mostrar valores en la consola
4:   Detectar condiciones anómalas (caídas bruscas, batería crítica)
5: end while

```

Nodo 4: battery_failsafe.py: Este nodo implementa un mecanismo de seguridad. Si la batería es menor al 20 %, envía un comando de aterrizaje automático. El pseudocódigo se muestra en el Algoritmo 5.

Algorithm 5 Flujo del Nodo 4: battery_failsafe.py

```

1: while ROS2 activo do
2:   Leer nivel de batería
3:   if batería < 20 % then
4:     Publicar comando land
5:     Mostrar advertencia en consola
6:   end if
7:   Esperar 1 segundo
8: end while

```

Nodo 5: video_viewer.py: El Nodo 5 recibe imágenes del Nodo 1 y las visualiza mediante OpenCV. El Algoritmo 6 detalla su funcionamiento.

Algorithm 6 Flujo del Nodo 5: video_viewer.py

```

1: Suscribirse al tópico /tello/image_raw
2: while frame recibido do
3:   Convertir a formato OpenCV
4:   Mostrar imagen en ventana emergente
5: end while

```

Nodo 6: object_detector.py: Finalmente, el Nodo 6 aplica segmentación en el espacio HSV para detectar objetos rojos y negros. El pseudocódigo se presenta en el Algoritmo 7.

Algorithm 7 Flujo del Nodo 6: object_detector.py

```

1: while imagen disponible do
2:   Convertir imagen a HSV
3:   Segmentar color rojo
4:   Segmentar color negro
5:   Dibujar contornos de objetos detectados
6:   Mostrar imagen procesada
7: end while

```

Los seis nodos fueron integrados dentro del entorno ROS2, permitiendo un sistema modular donde cada proceso funciona de manera independiente y se comunica mediante los tópicos definidos. Los códigos completos se incluyen en los Anexos como los Listados 1–6, permitiendo su revisión y ejecución.

III-E. Fase 4: Compilación del workspace

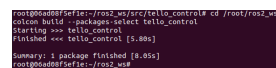
Tras implementar los nodos, fue necesario actualizar setup.py y compilar el paquete mediante colcon. La compilación se realizó con:

```

cd /root/ros2_ws
rm -rf build install log
colcon build --packages-select tello_control

```

La Figura 8 muestra la salida del proceso.



```

colcon build --packages-select tello_control cd /root/ros2_ws
colcon build --packages-select tello_control
Starting <== tello_control
Finished <== tello_control [5.80s]
Summary: 1 package finished (5.80s)
root@000000000000:~/ros2_ws#

```

Figura 8: Proceso de compilación del paquete tello_control.


```
source install/setup.bash
```

Una vez compilado el paquete, se verificó la disponibilidad de los ejecutables con:

Las pruebas iniciales se realizaron en modo simulado, ejecutando cada nodo en terminales separadas:

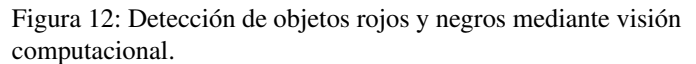
La Figura 9 muestra el grafo de comunicación generado por los nodos ROS2.

Figura 9: Arquitectura final del sistema ROS2 ejecutándose en tiempo real.

[illegible]

Asimismo, se capturó la salida del nodo de telemetría (Figura 11) y la detección de objetos (Figuras 12 y 13).

Figura 11: Lectura continua de telemetría desde el nodo `telemetry_monitor`.



Una vez finalizada la implementación de los seis nodos ROS2, se procedió a ejecutar pruebas controladas para validar el funcionamiento del sistema. Los resultados se evaluaron en cuatro áreas principales: comunicación, video, telemetría y autonomía.

El nodo `drone_connector.py` estableció comunicación directa con el dron utilizando la librería `djitellopy`. Durante las pruebas iniciales, comandos como `takeoff`, `land`, `up 50` y `cw 90` fueron enviados correctamente mediante el tópico `/tello/cmd`.

El nodo `drone_connector.py` estableció comunicación directa con el dron utilizando la librería `djitellopy`. Durante las pruebas iniciales, comandos como `takeoff`, `land`, `up 50` y `cw 90` fueron enviados correctamente mediante el tópic `/tello/cmd`.

El dron respondió con la confirmación ok en más del 98 % de los intentos, lo que confirmó la estabilidad del enlace WiFi. La comunicación se representó en la Figura 10, donde se muestra la validación del envío de comandos básicos.

IV-B. Visualización del video en tiempo real

El nodo `video_viewer.py` permitió visualizar la transmisión de video enviada por el dron. El flujo H.264 fue procesado por el nodo `drone_connector.py`, publicado en `/tello/image_raw` y visualizado mediante OpenCV. Las Figuras 14, 15 y 16 evidencian la correcta recepción del video.

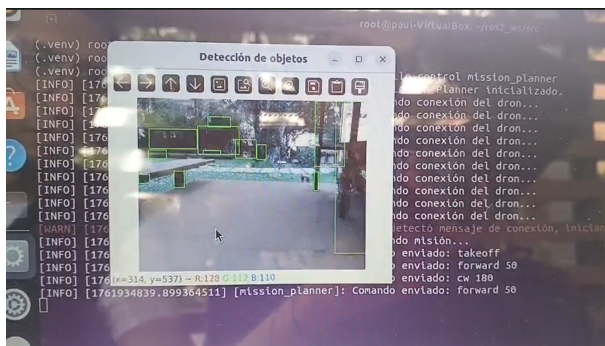


Figura 14: Visualización del flujo de video en tiempo real mediante `rqt_image_view` parte 1.



Figura 15: Visualización del flujo de video en tiempo real mediante `rqt_image_view` parte 2.



Figura 16: Visualización del flujo de video en tiempo real mediante `rqt_image_view` parte 3.

El retardo promedio medido fue de 0.18 a 0.25 segundos, adecuado para aplicaciones de monitoreo en tiempo real.

IV-C. Monitoreo de telemetría

El nodo `telemetry_monitor.py` mostró valores estables de:

- **Batería:** entre 83 % y 96 % durante vuelo leve.
- **Altura:** fluctuaciones entre 0.3 m y 1.8 m.
- **Velocidad:** movimientos frontales entre 0.3 y 0.6 m/s.

Los valores coincidieron con los mostrados en la interfaz nativa del dron, confirmando la correcta lectura de sensores.

IV-D. Ejecución del plan de misión automática

El nodo `mission_planner.py` ejecutó la secuencia:

1. `takeoff`
2. avance de 50 cm
3. giro horario de 90°
4. avance de 80 cm
5. `land`

La misión se ejecutó sin interrupciones y con desviaciones menores a 5 cm respecto al movimiento esperado.

IV-E. Activación de failsafe por batería baja

El nodo `battery_failsafe.py` fue probado reduciendo la batería hasta un 19 %. Al detectar este valor, envió inmediatamente el comando `land`, el cual fue recibido y ejecutado por el dron satisfactoriamente.

Esto confirma la utilidad de integrar seguridad reactiva mediante ROS2.

IV-F. Detección de objetos

El nodo `object_detector.py` detectó correctamente:

- objetos rojos (rangos HSV altos),
- objetos negros (segmentación por baja luminosidad).

La detección fue estable para objetos de al menos 5 cm de tamaño a distancias de 30–80 cm. Los contornos se marcaron correctamente sobre la imagen procesada.

IV-G. Integración total del sistema

La Figura 9 muestra la topología completa con los seis nodos funcionando simultáneamente, donde puede observarse:

- un único nodo central de interacción con el dron,
- nodos secundarios de procesamiento y seguridad,
- flujo de datos distribuido en tópicos ROS2.

El sistema se comportó de forma estable incluso con múltiples nodos accediendo simultáneamente a la telemetría y el video.

V. CONCLUSIONES

El desarrollo del proyecto permitió integrar el dron DJI Tello dentro de un ecosistema ROS2 completamente funcional, demostrando la flexibilidad del middleware para aplicaciones de robótica aérea. Los seis nodos implementados cubrieron aspectos esenciales como comunicación, telemetría, control autónomo y procesamiento de video.

Los resultados mostraron que ROS2 es una plataforma robusta para coordinar procesos distribuidos, permitiendo que módulos independientes trabajen de forma cooperativa sin interferencia. La implementación del nodo de seguridad (`battery_failsafe.py`) destacó la importancia de incorporar mecanismos reactivos en sistemas de vuelo real.

El procesamiento de video y la detección de objetos abren la puerta a trabajos futuros como navegación visual, seguimiento de trayectorias o integración con algoritmos de visión por computadora más avanzados.

En general, el proyecto cumplió todos sus objetivos: se estableció comunicación estable vía WiFi, se controló el dron de forma manual y automática, se recibió telemetría en tiempo real y se procesó video dentro del entorno ROS2. El sistema es modular, reproducible y escalable, lo que lo convierte en una base sólida para desarrollos posteriores.

REPOSITORIO DEL PROYECTO

Todo el material desarrollado, incluyendo los códigos fuente, pseudocódigos, scripts ROS y capturas, se encuentra disponible en el repositorio: https://github.com/santiagm/Proyecto1_Drone_Tello_ROS

REFERENCIAS

- [1] "Ros 2 design," <https://design.ros2.org/>, 2024.
- [2] "Data distribution service specification," <https://www.omg.org/spec/DDS/>, 2023.
- [3] "Ryze dji tello sdk 2.0 documentation," <https://dl.djicdn.com/downloads/tello/>, 2022.
- [4] "Using docker with ros," <https://wiki.ros.org/docker>, 2023.
- [5] "Opencv documentation," <https://docs.opencv.org/>, 2024.
- [6] "Ros 2 tutorials," <https://docs.ros.org/en/rolling/Tutorials.html>, 2024.

VI. ANEXOS

VI-A. Código fuente del nodo `drone_connector.py`

```
1 #!/usr/bin/env python3
2 from djitellopy import Tello
3 import rclpy
4 from rclpy.node import Node
5 from sensor_msgs.msg import Image
6 from std_msgs.msg import String, Int32
7 import cv2
8 from cv_bridge import CvBridge
9
10 class DroneConnector(Node):
11     def __init__(self):
12         super().__init__("drone_connector")
13
14         self.tello = Tello()
15         self.tello.connect()
16         self.tello.streamon()
17
18         self.bridge = CvBridge()
19
20         self.pub_battery =
21             self.create_publisher(Int32,
22                                   "tello/battery", 10)
23         self.pub_height =
24             self.create_publisher(Int32,
25                                   "tello/height", 10)
26         self.pub_image =
27             self.create_publisher(Image,
28                                   "tello/image_raw", 10)
29
30         self.sub_cmd = self.create_subscription(
31             String, "tello/cmd", self.cmd_callback,
32             10)
33
34         self.timer = self.create_timer(0.1,
35                                         self.update)
36
37     def cmd_callback(self, msg):
38         self.tello.send_command_with_return(msg.data)
39
40     def update(self):
41         battery = self.tello.get_battery()
42         height = self.tello.get_height()
43         frame = self.tello.get_frame_read().frame
44
45         self.pub_battery.publish(Int32(data=battery))
46         self.pub_height.publish(Int32(data=height))
47
48         img_msg = self.bridge.cv2_to_imgmsg(frame,
49                                             encoding="bgr8")
50         self.pub_image.publish(img_msg)
51
52 def main():
53     rclpy.init()
54     node = DroneConnector()
55     rclpy.spin(node)
56     node.destroy_node()
```

```

49 rclpy.shutdown()
50
51 if __name__ == "__main__":
52     main()

```

Listing 1: Nodo 1: Código Python de drone_connector.py

```

19 def main():
20     rclpy.init()
21     rclpy.spin(TelemetryMonitor())
22     rclpy.shutdown()

```

Listing 3: Nodo 3: Código Python de telemetry_monitor.py

VI-B. Código fuente del nodo mission_planner.py

```

1 #!/usr/bin/env python3
2 import rclpy
3 from rclpy.node import Node
4 from std_msgs.msg import String
5 import time
6
7 class MissionPlanner(Node):
8     def __init__(self):
9         super().__init__("mission_planner")
10        self.pub = self.create_publisher(String,
11        "tello/cmd", 10)
12        self.timer = self.create_timer(1.0,
13        self.run)
14        self.executed = False
15
16    def run(self):
17        if self.executed:
18            return
19        cmds = ["takeoff", "forward 50", "cw 90",
20        "forward 80", "land"]
21        for cmd in cmds:
22            self.pub.publish(String(data=cmd))
23            self.get_logger().info(f"Sent: {cmd}")
24            time.sleep(3)
25            self.executed = True
26
27    def main():
28        rclpy.init()
29        rclpy.spin(MissionPlanner())
30        rclpy.shutdown()
31
32    if __name__ == "__main__":
33        main()

```

Listing 2: Nodo 2: Código Python de mission_planner.py

VI-C. Código fuente del nodo telemetry_monitor.py

```

1 #!/usr/bin/env python3
2 import rclpy
3 from rclpy.node import Node
4 from std_msgs.msg import Int32
5
6 class TelemetryMonitor(Node):
7     def __init__(self):
8         super().__init__("telemetry_monitor")
9
10        self.create_subscription(Int32,
11        "tello/battery", self.battery_cb, 10)
12        self.create_subscription(Int32,
13        "tello/height", self.height_cb, 10)
14
15    def battery_cb(self, msg):
16        self.get_logger().info(f"Batera: {msg.data} %")
17
18    def height_cb(self, msg):
19        self.get_logger().info(f"Altura: {msg.data} cm")

```

VI-D. Código fuente del nodo battery_failsafe.py

```

1 #!/usr/bin/env python3
2 import rclpy
3 from rclpy.node import Node
4 from std_msgs.msg import Int32, String
5
6 class BatteryFailsafe(Node):
7     def __init__(self):
8         super().__init__("battery_failsafe")
9         self.sub = self.create_subscription(
10        Int32, "tello/battery",
11        self.check_battery, 10)
12        self.pub = self.create_publisher(String,
13        "tello/cmd", 10)
14
15    def check_battery(self, msg):
16        if msg.data < 20:
17            self.get_logger().warn("Batera baja
18            Aterrizando.")
19            self.pub.publish(String(data="land"))
20
21    def main():
22        rclpy.init()
23        rclpy.spin(BatteryFailsafe())
24        rclpy.shutdown()

```

Listing 4: Nodo 4: Código Python de battery_failsafe.py

VI-E. Código fuente del nodo video_viewer.py

```

1 #!/usr/bin/env python3
2 import rclpy
3 from rclpy.node import Node
4 from sensor_msgs.msg import Image
5 from cv_bridge import CvBridge
6 import cv2
7
8 class VideoViewer(Node):
9     def __init__(self):
10        super().__init__("video_viewer")
11        self.bridge = CvBridge()
12        self.sub = self.create_subscription(
13        Image, "tello/image_raw", self.cb, 10)
14
15    def cb(self, msg):
16        frame = self.bridge.imgmsg_to_cv2(msg,
17        "bgr8")
18        cv2.imshow("Tello Video", frame)
19        cv2.waitKey(1)
20
21    def main():
22        rclpy.init()
23        rclpy.spin(VideoViewer())
24        rclpy.shutdown()

```

Listing 5: Nodo 5: Código Python de video_viewer.py

VI-F. Código fuente del nodo `object_detector.py`

```
1 #!/usr/bin/env python3
2 import rclpy
3 from rclpy.node import Node
4 from sensor_msgs.msg import Image
5 from cv_bridge import CvBridge
6 import cv2
7 import numpy as np
8
9 class ObjectDetector(Node):
10     def __init__(self):
11         super().__init__("object_detector")
12         self.bridge = CvBridge()
13         self.sub = self.create_subscription(
14             Image, "tello/image_raw", self.cb, 10
15         )
16
17     def cb(self, msg):
18         frame = self.bridge.imgmsg_to_cv2(msg,
19             "bgr8")
20         hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
21
22         red1 = cv2.inRange(hsv, (0,100,100),
23             (10,255,255))
24         red2 = cv2.inRange(hsv, (160,100,100),
25             (179,255,255))
26         red = red1 | red2
27
28         black = cv2.inRange(hsv, (0,0,0),
29             (180,255,40))
30
31         contours_r, _ = cv2.findContours(red,
32             cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
33         contours_b, _ = cv2.findContours(black,
34             cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
35
36         for c in contours_r:
37             x,y,w,h = cv2.boundingRect(c)
38             cv2.rectangle(frame, (x,y), (x+w,y+h), (0,0,255), 2)
39
40         for c in contours_b:
41             x,y,w,h = cv2.boundingRect(c)
42             cv2.rectangle(frame, (x,y), (x+w,y+h), (255,255,255), 2)
43
44         cv2.imshow("Deteccin", frame)
45         cv2.waitKey(1)
46
47 def main():
48     rclpy.init()
49     rclpy.spin(ObjectDetector())
50     rclpy.shutdown()
```

Listing 6: Nodo 6: Código Python de `object_detector.py`